



RCA & RUNBOOK

VIRTUAL CONTROL
VMWARE CLOUD FOUNDATION SOLUTIONS

Fleet ↔ vCenter Trust Cascade Repair

Multi-layer recovery after vCenter cert rotation cascaded through Fleet's deployment target, K8s platform-trust bundle, locker, and auto-generated service account.

VCENTER CERT

PLATFORM-TRUST

SVC-OPS-VC

NDCHELPER

INVENTORY RESYNC

VCF 9.0.1

VMware Cloud Foundation

Fleet ↔ vCenter Trust Cascade Repair — Multi-Layer Recovery After vCenter Cert Rotation

Prepared by: Virtual Control LLC **Date:** May 18, 2026 **Document Version:** 1.0 **Classification:** Internal — Lab Environment **VCF Version:** 9.0.1.0 **Fleet Management:** vmlcm-service-9.0.1.0-SNAPSHOT **VCF Operations:** 9.0.1.0.24960351 **vCenter:** 9.0.1.0.24957454

Scope. This document is the standalone runbook for the **multi-layer trust cascade** that surfaces when a vCenter certificate has been rotated *after* Fleet/LCM pinned the prior thumbprint. Symptoms cascade through four discrete stores, each fixable independently but only resolvable together. Companion document for the cert-rotation-on-Fleet-side variant (which surfaces as "VCF Operations Fleet Management is Not Ready") is [Fleet-Trust-Recovery-After-Cert-Rotation-KB402063-2026-05-17.md](#). Broader integration RCA (instance-limit, NPE-flood, DB cleanup) is [Fleet-Operations-Integration-Recovery-RCA.md](#).

Use this document when:

- A vCenter certificate has been rotated in the recent past (days to weeks)
- Fleet's deployment target shows an outdated `vcCertificate` PEM
- Import workflows fail with `LCMCOMMON80062` (cert) → `LCMCOMMON80006` / `LCMCOMMON80015` (InvalidLogin) → `LCMCOMMON80018` (NoPermission) — a moving target of error codes that all trace back to the same root cause
- The `svc-account` name in Fleet's deployment target doesn't match any user that actually exists in vCenter SSO

Table of Contents

1. Executive Summary
2. Environment Reference
3. Symptom Timeline (Cascade)
4. Root Cause Analysis (Four Trust Stores)
 - 4.1 Layer 1 — Fleet Deployment Target `vcCertificate`
 - 4.2 Layer 2 — K8s `platform-trust` Secret (NDC Trust Bundle)
 - 4.3 Layer 3 — Fleet Locker vCenter Password Reference
 - 4.4 Layer 4 — Auto-Generated `svc-ops-vc-*` Account
5. Recovery Procedure (Verbatim Commands)
 - 5.1 Phase 1 — Snapshots (Mandatory)
 - 5.2 Phase 2 — Fetch and Compare Thumbprints
 - 5.3 Phase 3 — Patch Fleet Deployment Target via PUT
 - 5.4 Phase 4 — Patch K8s `platform-trust` Secret
 - 5.5 Phase 5 — New Locker Password + Re-point Deployment Target
 - 5.6 Phase 6 — Create the `svc-ops-vc-*` User in vCenter SSO
 - 5.7 Phase 7 — Grant Administrator Role with Propagate
 - 5.8 Phase 8 — Trigger Inventory Resync
6. Verification Matrix

- 7. Rollback Procedure
- 8. KB Citation Map
- 9. Incident Log — 2026-05-18 (First Documented Use)
- 10. Lessons Learned
- 11. Document Information

1. Executive Summary

When a vCenter Server certificate is rotated *after* a Fleet/LCM appliance has registered it as a Deployment Target, the trust state breaks across **four discrete stores** that each need explicit repair:

1. **Fleet** `vr1cm` **REST API**'s deployment-target `vcCertificate` field (the LCM-side view of the cert)
2. **K8s** `platform-trust` **Secret** in the `vmosp-platform` namespace (the NDC trust bundle that `vr1cm-server.service` reads via `NdcHelper.pullCertificate` keyed by SHA-256)
3. **Fleet locker password entry** referenced by `vcPassword` (typically tied to an old service account whose password no one knows)
4. **vCenter SSO**'s user database (which may be missing an auto-generated `svc-ops-vc-*` account that Fleet's workflow expects to exist)

Each layer surfaces a *different error code* during retry, leading to the misleading impression that fixes for prior layers regressed. They didn't — each retry just exposed the next layer down. Recovery requires touching all four stores in order, then triggering an inventory resync from the Fleet UI to validate.

Root Cause Statement. vCenter cert rotation invalidates Fleet's pinned thumbprint at the vmomi level. Restoring trust requires synchronized updates to the LCM-side metadata, K8s trust bundle, locker credentials, and SSO user database. No public Broadcom KB covers all four layers; KB 402063 covers (1), KB 421692 covers metadata refresh in general, and the rest are inferred from runtime error messages and the Fleet appliance's K8s topology (`kind-bootstrap-control-plane` + `vmosp-platform` namespace running trust-manager).

Fix outline. Patch (1) via `PUT /1cm/lcops/api/v2/datacenters/{dc}/vcenters/{host}`. Patch (2) via `kubectl patch secret` inside the `kind-bootstrap-control-plane` container. Replace (3) by creating a new locker entry and re-pointing `vcPassword`. Create (4) manually in vSphere Client → SSO → Users and Groups, granting the Administrator role with **Propagate to children**. Then trigger inventory resync from Fleet UI → Components → operations → **MANAGE** → **TRIGGER INVENTORY SYNC**.

2. Environment Reference

Component	Value (lab)
vCenter	<code>vcenter.lab.local</code> — 192.168.1.69 — Build 24957454 (cert rotated 2026-05-13)
Fleet Management (LCM)	<code>lcm.lab.local</code> — 192.168.1.95 — Photon 5, hybrid Java + KIND/Docker

Component	Value (lab)
VCF Operations	<code>vcf-ops.lab.local</code> — 192.168.1.77
VCF Ops collector	<code>collector.lab.local</code> — 192.168.1.79
Fleet K8s namespace (NDC)	<code>vmsp-platform</code>
Fleet trust bundle (source Secret)	<code>platform-trust</code> (label <code>trust.vmsp.vmware.com/bundle=platform-trust</code>)
Fleet trust bundle (target ConfigMap)	<code>platform-trust</code> with key <code>bundle.pem</code> (auto-synced by cert-manager trust-manager)
Fleet vrlcm DB	Postgres on 127.0.0.1:5432, db <code>vrlcm</code> , user <code>postgres / vmware</code> , app user/db <code>vrlcm / vrlcm</code>
Fleet break-glass UI flag	<code>touch /var/lib/vrlcm/UI_ENABLED</code>
Fleet Java service	<code>vrlcm-server.service</code> (PID 1244, port 8080 internal); JAR <code>vmlcm-service-9.0.1.0-SNAPSHOT.jar</code> ; truststore <code>/var/lib/vrlcm/bundle/fips/cacerts.bcfks</code> (BCFKS, pw <code>changeit</code>)
KIND container	<code>kind-bootstrap-control-plane</code> — hosts inner K8s cluster + 27 <code>vmsp-platform</code> pods

3. Symptom Timeline (Cascade)

Time (UTC)	Surface	Error / State	Root cause layer
Steady state	Fleet UI → Components → operations	<code>Pending Import</code> (UI badge); cert thumbprint mismatch error masked	All four layers misaligned
Trigger Import → Precheck	wizard Precheck	<code>LCMCOMMON80062</code> — <code>CertificateValidationException: thumbprint doesn't match</code>	Layer 1 (Fleet deployment target)
After patching Layer 1 via PUT	Retry	<code>LCMCOMMON80062</code> (same code, deeper in stack: <code>NdcHelper.pullCertificate</code> — <code>No certificate found in trust store for key id <sha256></code>)	Layer 2 (K8s <code>platform-trust</code> Secret)
After patching Layer 2	Retry	<code>LCMCOMMON80006 / LCMCOMMON80015</code> — <code>vim.fault.InvalidLogin: Cannot complete login due to an incorrect username or password</code>	Layer 3 (locker pwd doesn't match SSO user)
After creating new locker entry + repointing	Retry	<code>LCMCOMMON80018</code> — <code>vim.fault.NoPermission: Permission to perform this operation was denied</code> (this is also where Fleet auto-rewrites <code>vcUsername</code> to <code>svc-ops-vc-<uuid>-<uuid>-1@vsphere.local</code>)	Layer 4 (SSO user missing / lacks role)
After creating SSO user + Administrator role	Trigger inventory resync	Updating Environment request status to <code>COMPLETED</code> for request type : <code>VC_DATA_COLLECTION</code>	All layers aligned

Misleading retry signal. Each layer's fix appears at first to *cause* the next layer's error to surface. It doesn't. The next-layer error was always going to be the next step in the validation flow — you just couldn't see it until the prior step started

passing. Resist the urge to revert a fix when retry surfaces a new error code.

4. Root Cause Analysis (Four Trust Stores)

4.1 Layer 1 — Fleet Deployment Target `vcCertificate`

Stored in Fleet's `vr1cm` Postgres DB and exposed by:

```
GET /lcm/lcops/api/v2/datacenters/DEFAULT_DC/vcenters
```

Response includes a `vcCertificate` field containing a full PEM. This PEM is the cert Fleet **expected** vCenter to present. When vCenter's cert is rotated, this PEM is now stale and every TLS handshake initiated by `vr1cm-server` against vCenter fails at the vmomi thumbprint verifier.

This is the *first* layer because the LCM workflow probes it directly before any K8s-side check.

4.2 Layer 2 — K8s `platform-trust` Secret (NDC Trust Bundle)

Inside the `kind-bootstrap-control-plane` Docker container, a cert-manager `trust-manager` Bundle named `platform-trust` watches Secrets with label `trust.vmsp.vmware.com/bundle=platform-trust` and assembles them into a ConfigMap `platform-trust` (key `bundle.pem`) plus a JKS/P12 sibling. The Secret with that label, also named `platform-trust` in `vmsp-platform`, has a `ca.crt` field that holds the trusted vCenter cert PEM.

Fleet's Java side reads this trust bundle via `com.vmware.vrealize.lcm.util.NdcHelper.pullCertificate(kid)` — **lookup is keyed by the SHA-256** of the cert ("kid"). If the live cert's SHA-256 is not in the bundle, the lookup fails with:

```
java.lang.RuntimeException: No certificate found in trust store for kid <sha256>
    at com.vmware.vrealize.lcm.util.NdcHelper.pullCertificate(NdcHelper.java:78)
```

Important: **patching Layer 1 (the deployment-target metadata) does NOT propagate to Layer 2.** `trust-manager` re-syncs the ConfigMap from the Secret only; nothing pushes from the deployment-target field into the Secret. The two stores are independent.

4.3 Layer 3 — Fleet Locker vCenter Password Reference

The deployment target's `vcPassword` field is not a raw password — it's a reference string:

```
locker:password:<vmid>:<alias>
```

Pointing at a Fleet locker entry. The locker entry holds `userName` (the SSO account) and `password` (encrypted; never readable via API). When the original locker entry was created, both `userName` and `password` matched a vCenter SSO account that existed at that time. If either has changed since (e.g., svc-account auto-rotation, vCenter password reset, account deletion), the locker entry is stale and `Connect-VIServer` from vmomi fails with `InvalidLogin`.

The locker entry **cannot be edited in place** if it is referenced by other items:

```
{"status":"","message":"Password associated with other items can not be edited in place.",
"errorCode":"LCM_PASSWORD_API_ERROR0000"}
```

So the fix is to create a *new* locker entry and re-point the deployment target's `vcPassword` field to it.

4.4 Layer 4 — Auto-Generated `svc-ops-vc-*` Account

When Fleet's import workflow runs against a deployment target, it generates a **unique service-account name** in the form:

```
svc-ops-vc-<uuid1>-<uuid2>-1@vsphere.local
```

Example observed: `svc-ops-vc-1037eae3-5450-4ed0-91b2-86051f73be91-62603f69f72042de-1@vsphere.local`

It expects this account to exist in vCenter SSO. On a clean first-time deployment, the workflow may create the account itself; on a *retry* after partial state, it may bypass creation and try to log in directly — producing the `Invalid Login` error described in §4.3, but with a *different* username than what the locker entry references.

Two distinct subproblems within this layer:

1. **Username drift.** Each retry can rewrite the deployment target's `vcUsername` to a new auto-generated `svc-` account name, overwriting any manual edits made by the operator.
2. **Permission scoping.** Even when the account exists, it must be granted the **Administrator** role at the **Global Permissions** scope with **Propagate to children** — without propagate, child-object inventory reads fail with `im.fault.NoPermission` even though login succeeds.

The fastest reliable fix is to manually create the auto-generated username in vSphere SSO, set the password to a known value matching the locker entry, and grant Administrator + Propagate. This wins the race against retry rewrites because the next retry finds the account already exists and proceeds.

5. Recovery Procedure (Verbatim Commands)

5.1 Phase 1 — Snapshots (Mandatory)

```
# Run from Windows workstation
Set-PowerCLIConfiguration -InvalidCertificateAction Ignore -Confirm:$false -Scope Session | Out-Null
$cred = New-Object System.Management.Automation.PSCredential('administrator@vsphere.local',
    (ConvertTo-SecureString '<vc-admin-password>' -AsPlainText -Force))
Connect-VIServer -Server 192.168.1.69 -Credential $cred -Force

$ts = (Get-Date -Format 'yyyy-MM-dd')
foreach ($name in 'fleet','vcf-ops','collector') {
    New-Snapshot -VM $name -Name "pre-trust-cascade-$ts" `
        -Memory:$false -Quiesce:$false -Confirm:$false
}
```

Use `-Quiesce:$false`. Quiesced snapshots on Fleet's kind-container guest deadlock at 99% (see [\[\[reference-fleet-quiesce-deadlock\]\]](#) / sibling RCA §15.4).

5.2 Phase 2 — Fetch and Compare Thumbprints

```
$tcp = [System.Net.Sockets.TcpClient]::new('192.168.1.69',443)
$ssl = [System.Net.Security.SslStream]::new($tcp.GetStream(),$false,({$true}))
$ssl.AuthenticateAsClient('192.168.1.69')
$c = [System.Security.Cryptography.X509Certificates.X509Certificate2]::new($ssl.RemoteCertificate)
```

```
$sha1 = $c.Thumbprint
$sha256bytes = ([System.Security.Cryptography.SHA256]::Create()).ComputeHash($c.RawData)
$sha256 = -join ($sha256bytes | ForEach-Object { $_.ToString('X2') })
"SHA-1 : $sha1"
"SHA-256 : $sha256"
$ssl.Close(); $tcp.Close()
```

```
# On Fleet – fetch what the deployment target THINKS vCenter is
curl -sk -u admin@local:'<password>' \
  https://localhost/lcm/lcops/api/v2/datacenters/DEFAULT_DC/vcenters \
  | python3 -c "import sys,json; print(json.load(sys.stdin)[0]['vcCertificate'])" \
  > /tmp/stored-vc.pem
openssl x509 -in /tmp/stored-vc.pem -noout -fingerprint -sha1 -dates

# Compare with live
echo | openssl s_client -connect 192.168.1.69:443 -servername vcenter.lab.local 2>/dev/null \
  | openssl x509 -noout -fingerprint -sha1 -dates
```

Confirm the two SHA-1s differ. If they match, Layer 1 is not the problem — jump to §5.4.

5.3 Phase 3 — Patch Fleet Deployment Target via PUT

```
# Run on Fleet – fetch live PEM
echo | openssl s_client -connect 192.168.1.69:443 -servername vcenter.lab.local 2>/dev/null \
  | openssl x509 > /tmp/live-vc.pem

# Build full PUT body. ALL fields must be present – empty PUT wipes them.
python3 <<'EOF'
import json
with open('/tmp/live-vc.pem') as f:
    live = f.read().strip()
payload = {
    "vCenterName": "vcenter.lab.local",
    "vCenterHost": "vcenter.lab.local",
    "vcUsername": "administrator@vsphere.local", # temporarily known-good
    "vcPassword": "locker:password:<vmid>:<alias>", # will replace in §5.5
    "vcUsedAs": "MANAGEMENT",
    "vcCertificate": live,
    "sddcManagerHostname": "sddc-manager.lab.local",
    "vcVersion": "9.0.1.0.24957454"
}
with open('/tmp/dt.json','w') as f:
    json.dump(payload, f, indent=2)
EOF

curl -sk -u admin@local:'<password>' \
  -X PUT \
  -H 'Content-Type: application/json' \
  -d @/tmp/dt.json \
  -w "\nHTTP %{http_code}\n" \
  "https://localhost/lcm/lcops/api/v2/datacenters/DEFAULT_DC/vcenters/vcenter.lab.local"
# Expected: {"requestId":"..."} + HTTP 200
```

Do not send `{}` as the PUT body. Doing so will return HTTP 200 with a requestId but will **wipe optional fields** (`vcUsername`, `vcPassword`, `vcVersion`, `sddcManagerHostname` all become null). Always send the complete object.

Verify:

```

curl -sk -u admin@local:'<password>' \
  "https://localhost/lcm/lcops/api/v2/datacenters/DEFAULT_DC/vcenters" \
  | python3 -c "import sys,json; print(json.load(sys.stdin)[0]['vcCertificate'])" \
  | openssl x509 -noout -fingerprint -sha1
# Must match the live SHA-1 from §5.2

```

5.4 Phase 4 — Patch K8s `platform-trust` Secret

```

# On Fleet – verify the Secret has the right label
docker exec kind-bootstrap-control-plane \
  kubectl get secret platform-trust -n vmsp-platform -o jsonpath='{.metadata.labels}'
# Expect: {"trust.vmsp.vmware.com/bundle":"platform-trust"}

# Show all Secrets that match the bundle source selector
docker exec kind-bootstrap-control-plane \
  kubectl get secret -A -l trust.vmsp.vmware.com/bundle=platform-trust

# Patch ca.crt with live cert (base64)
B64=$(base64 -w0 /tmp/live-vc.pem)
docker exec kind-bootstrap-control-plane \
  kubectl patch secret platform-trust -n vmsp-platform --type=json \
  -p="[{\"op\":\"replace\",\"path\":\"/data/ca.crt\",\"value\":\"$B64\"}]"

# Verify
docker exec kind-bootstrap-control-plane \
  kubectl get secret platform-trust -n vmsp-platform -o jsonpath='{.data.ca.crt}' \
  | base64 -d \
  | openssl x509 -noout -fingerprint -sha1
# Must match live SHA-1

# Trust-manager auto-resyncs the Bundle to all namespaces – verify
docker exec kind-bootstrap-control-plane \
  kubectl get bundles.trust.cert-manager.io platform-trust \
  -o jsonpath='{.status.conditions[?(@.type=="Synced")].status}'
# Expect: True

```

5.5 Phase 5 — New Locker Password + Re-point Deployment Target

The existing locker entry referenced by `vcPassword` is **edit-locked** if used by other items (LCM_PASSWORD_API_ERROR0000). Workaround: create a new entry, point the deployment target at it.

```

# POST new locker entry
cat > /tmp/newpw.json <<'EOF'
{
  "alias": "VCF-vcenter.lab.local-adminPassword",
  "userName": "administrator@vsphere.local",
  "password": "<vc-admin-password>",
  "passwordDescription": "vCenter SSO admin – replacement for stale svc-account reference"
}
EOF
RESP=$(curl -sk -u admin@local:'<password>' \
  -X POST -H 'Content-Type: application/json' -d @/tmp/newpw.json \
  "https://localhost/lcm/locker/api/v2/passwords")
echo "$RESP"
# Expect: {"vmid":"<NEW-UUID>","alias":"VCF-vcenter.lab.local-adminPassword",...}

NEW_VMID=$(echo "$RESP" | python3 -c "import json,sys; print(json.load(sys.stdin)['vmid'])")

# Re-issue the deployment target PUT with the new locker reference
python3 <<PYEOF
import json
with open('/tmp/live-vc.pem') as f: live = f.read().strip()
payload = {
  "vCenterName": "vcenter.lab.local",

```



```

    "vCenterHost": "vcenter.lab.local",
    "vcUsername": "administrator@vsphere.local",
    "vcPassword": "locker:password:${NEW_VMID}:VCF-vcenter.lab.local-adminPassword",
    "vcUsedAs": "MANAGEMENT",
    "vcCertificate": live,
    "sddcManagerHostname": "sddc-manager.lab.local",
    "vcVersion": "9.0.1.0.24957454"
}
with open('/tmp/dt2.json','w') as f: json.dump(payload, f, indent=2)
PYEOF

curl -sk -u admin@local:'<password>' \
-X PUT -H 'Content-Type: application/json' -d @/tmp/dt2.json \
-w "\nHTTP %{http_code}\n" \
"https://localhost/lcm/lcops/api/v2/datacenters/DEFAULT_DC/vcenters/vcenter.lab.local"

```

On next retry, Fleet's workflow may auto-rewrite `vcUsername` away from `administrator@vsphere.local` to an auto-generated `svc-ops-vc-<uuid>-<uuid>-1@vsphere.local`. This is expected behavior on retry from a partial state — do not try to "fight" it by repeatedly setting `vcUsername` back. Instead, accept the auto-generated name and proceed to §5.6 to create the corresponding user in vCenter SSO.

5.6 Phase 6 — Create the `svc-ops-vc-*` User in vCenter SSO

After triggering retry once (see §5.8), capture the auto-generated username from the deployment target:

```

curl -sk -u admin@local:'<password>' \
  "https://localhost/lcm/lcops/api/v2/datacenters/DEFAULT_DC/vcenters" \
  | python3 -c "import sys,json; print(json.load(sys.stdin)[0]['vcUsername'])"
# Example: svc-ops-vc-1037eae3-5450-4ed0-91b2-86051f73be91-62603f69f72042de-1@vsphere.local

```

In vSphere Client (signed in as `administrator@vsphere.local`):

1. **Menu** → **Administration** → **Single Sign On** → **Users and Groups**
2. **Users** tab → **Domain** dropdown = `vsphere.local`
3. + **ADD USER**
4. Fields:

Field	Value
Username	the captured <code>svc-ops-vc-<uuid>-<uuid>-1</code> (without <code>@vsphere.local</code>)
Password	match the locker entry password (e.g. <code><vc-admin-password></code> if reusing)
Confirm Password	same
First Name	VCF Ops
Last Name	Service Account
5. Add / Save	

Verify login works from Fleet:

```

# SOAP login probe – confirms the SSO user can authenticate
curl -sk -X POST "https://192.168.1.69/sdk" \
-H "Content-Type: text/xml; charset=utf-8" \
-H "SOAPAction: \"urn:vim25/9.0.0.0\"" \

```

```
-d '<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:urn="urn:vim25">
  <soapenv:Body><urn:Login>
    <urn:_this type="SessionManager">SessionManager</urn:_this>
    <urn:userName>svc-ops-vc-<uuid>-<uuid>-1@vsphere.local</urn:userName>
    <urn:password><vc-admin-password></urn:password>
  </urn:Login></soapenv:Body>
</soapenv:Envelope>' | head -c 400
# Look for: <LoginResponse> with userName and loginTime – that's success.
```

5.7 Phase 7 — Grant Administrator Role with Propagate

In vSphere Client:

1. **Menu** → **Administration** → **Access Control** → **Global Permissions**
2. + **ADD** permission
3. Domain: `vsphere.local`
4. Search for `svc-ops-vc-<uuid>` → select
5. **Role: Administrator** (NOT any "-administrator (sample)" variant — those are read-only)
6. **Propagate to children: ✓ checked**
7. **OK / Save**

Verify inventory permission from Fleet:

```
# After SOAP login (cookie-jar from §5.6), retrieve datacenter list
curl -sk -b /tmp/cookies.txt -X POST "https://192.168.1.69/sdk" \
-H "Content-Type: text/xml; charset=utf-8" \
-H "SOAPAction: \"urn:vim25/9.0.0.0\"" \
-d '<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:urn="urn:vim25">
  <soapenv:Body><urn:RetrievePropertiesEx>
    <urn:_this type="PropertyCollector">propertyCollector</urn:_this>
    <urn:specSet>
      <urn:propSet><urn:type>Datacenter</urn:type><urn:pathSet>name</urn:pathSet></urn:propSet>
      <urn:objectSet>
        <urn:obj type="Folder">group-d1</urn:obj>
        <urn:selectSet xsi:type="urn:TraversalSpec" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
          <urn:name>visitFolders</urn:name><urn:type>Folder</urn:type>
          <urn:path>childEntity</urn:path><urn:skip>false</urn:skip>
        </urn:selectSet>
      </urn:objectSet>
    </urn:specSet><urn:options/>
  </urn:RetrievePropertiesEx></soapenv:Body>
</soapenv:Envelope>' | head -c 400
# Look for: val xsi:type="xsd:string">DATACENTER-NAME – that's success.
# If "NoPermission" appears, role grant did not save with Propagate.
```

5.8 Phase 8 — Trigger Inventory Resync

In Fleet UI (with break-glass UI enabled per §5 of the sibling KB 402063 doc):

1. **Operations Fleet Management** → **Components**
2. Click the `operations` row → **MANAGE** button
3. Top toolbar → **TRIGGER INVENTORY SYNC** (the leftmost circular-arrow button)
4. Watch `/var/log/vr1cm/vmware_vr1cm.log` on Fleet:

```
tail -F /var/log/vr1cm/vmware_vr1cm.log | grep -iE
"snapshotInventoryupdate|VC_DATA_COLLECTION|COMPLETED|FAILED"
```

Success log marker:

```
<timestamp> INFO vrlcm[<pid>] [scheduling-1] [c.v.v.l.r.c.RequestProcessor]
-- Updating the Environment request status to COMPLETED for request type : VC_DATA_COLLECTION.
```

6. Verification Matrix

After Phase 8 completes, verify each layer:

Layer	Check	Expected
1 — Deployment target	<code>GET /lcm/lcops/api/v2/datacenters/DEFAULT_DC/vcenters → vcCertificate fingerprint</code>	Matches live vCenter SHA-1
2 — K8s platform-trust	<code>kubectl get secret platform-trust -n vmosp-platform -o jsonpath='{.data.ca\.crt}' → decode → SHA-1</code>	Matches live vCenter SHA-1
3 — Locker entry	<code>GET /lcm/locker/api/v2/passwords/<new-vmid></code>	userName matches expected SSO user; entry exists
4 — SSO user	SOAP login probe from Fleet	Returns <code>LoginResponse</code> with userName
4 — SSO role	SOAP RetrievePropertiesEx for Datacenter	Returns at least one Datacenter, no NoPermission
Env state	<code>GET /lcm/lcops/api/v2/environments/<envId></code>	<code>environmentStatus: COMPLETED, syncedEnvironmentStatus: COMPLETED, fleetEnabled: true</code>
Fleet UI	Components page	operations row state changes from "Deployment Failed" / "New Deployment" to a clean state with MANAGE only
Log	<code>/var/log/vrlcm/vmware_vrlcm.log</code>	No new <code>CertificateValidationException</code> , <code>InvalidLogin</code> , or <code>NoPermission</code> since the last fix phase

7. Rollback Procedure

If verification fails at any phase, roll back to the snapshots taken in §5.1:

```
foreach ($name in 'fleet','vcf-ops','collector') {
  $vm = Get-VM -Name $name
  $snap = Get-Snapshot -VM $vm -Name "pre-trust-cascade-*"
  if ($snap) { Set-VM -VM $vm -Snapshot $snap -Confirm:$false }
}
```

After revert, the prior state (with all four layers misaligned) returns. From there, either: - Re-attempt this runbook with corrected inputs - Reach for the broader [Fleet-Operations-Integration-Recovery-RCA.md](#) for Phase-5 DB cleanup approaches

Snapshots from the 2026-05-17 KB 402063 recovery may still exist; check `Get-Snapshot -VM fleet` before consolidating.

8. KB Citation Map

KB	Layer it addresses	Notes
402063	Layer 1 partially (DELETE/recreate env approach)	Public; covers env-level cleanup. See sibling KB402063 doc.
421692	Stale workflow-JSON thumbprint (similar mechanism, different scope)	Public; VCF Installer flow, not Fleet runtime
424770	SDDC Manager workflow JSON patch	Public; same pattern as 421692
432320	Root-cause framing for cert-rotation cascade	Public
430561	Disconnect → enable UI → delete → reconnect	Public; UI procedure outline
322036	NSX-T Compute Manager thumbprint refresh	Public; analogous mechanism on NSX side
323341	NSX CM "Down" after vCenter cert replace	Public; similar
410260	Internal/support-only — same procedure as 402063	Restricted to Broadcom Support

None of the public KBs covers all four layers in a single document. This RCA fills that gap.

9. Incident Log — 2026-05-18 (First Documented Use)

Property	Value
Trigger	vCenter cert rotated on 2026-05-13 (new SHA-1 FC:EB:18:6E:E7:2F:A8:EA:B7:E1:AA:B7:E1:C3:64:4C:8A:52:90:A3 , valid through 2028-05-13). Stale stored cert was AE:48:6A:07:62:A5:39:24:1F:4A:1D:0E:30:4A:24:B2:4C:BF:71:C3 (issued 2026-05-02).
Stale env (Layer 1 victim)	97dee43f-95f9-4c04-ab3c-b9585d0ad2f2 (vrops operations env created during 2026-05-17 KB 402063 reconnect)
Locker entry replaced	Old: 1719a278-d8ec-40ac-b73c-1e3aa6746575:VCF-vcenter.lab.local-vcPassword (svc-lcm-vc-df1-1, unknown password). New: c7288743-6d9e-4c27-befd-51300f693750:VCF-vcenter.lab.local-adminPassword (administrator@vsphere.local)
Auto-generated svc account	svc-ops-vc-1037eae3-5450-4ed0-91b2-86051f73be91-62603f69f72042de-1@vsphere.local (manually created in vSphere SSO with password <vc-admin-password> , Administrator role + Propagate)
Inventory resync requestId pattern	snapshotInventoryupdate task; final log marker Updating the Environment request status to COMPLETED for request type : VC_DATA_COLLECTION at 2026-05-18T13:34:Z
Net hands-on time	~3.5 hours (most of it spent identifying Layer 2 and Layer 4)

Property	Value
Total elapsed (incl. dead-end probes)	~6 hours
Result	operations env fully integrated (envStatus: COMPLETED , fleetEnabled: True); VCFA deploy unblocked

Error code progression (each one fixed before the next surfaced):

Step	Error code	Layer
Initial precheck	LCMCOMMON80062 — CertificateValidationException (thumbprint)	Layer 1
After Layer 1 fix	LCMCOMMON80062 — NdcHelper "No certificate found in trust store for kid"	Layer 2
After Layer 2 fix	LCMCOMMON80006 / LCMCOMMON80015 — InvalidLogin	Layer 3
After Layer 3 fix	LCMCOMMON80018 — NoPermission	Layer 4
After Layer 4 fix	(no error — VC_DATA_COLLECTION COMPLETED)	All aligned

10. Lessons Learned

1. Four trust stores, not one. A vCenter cert rotation touches Layer 1 (LCM API metadata), Layer 2 (K8s trust bundle), Layer 3 (locker reference), and Layer 4 (SSO user) independently. There is no public Broadcom procedure that unifies all four. Plan to touch each one.

2. Each retry's error code is the *next* step failing, not a regression. When LCMCOMMON80062 changes to LCMCOMMON80006, the fix for 80062 worked — you just unblocked the validation step that reveals 80006.

3. NdcHelper.pullCertificate(kid) is keyed by SHA-256. The "kid" in stack traces like No certificate found in trust store for kid <hex> is the cert's SHA-256, not SHA-1 nor a user-supplied identifier. Match against the openssl x509 -fingerprint -sha256 output (lowercase, no colons).

4. Empty-PUT against the deployment-target endpoint wipes optional fields. PUT /lcm/lcops/api/v2/datacenters/{dc}/vcenters/{host} with body {} returns HTTP 200 with a requestId but nulls out vcUsername, vcPassword, vcVersion, sddcManagerHostname. Always send the complete object.

5. Fleet's PUT handler auto-refreshes vcCertificate from live vCenter. A side-effect of the PUT mechanism: even sending an empty body causes Fleet to re-fetch the cert. This is the *only* mechanism that updates Layer 1 short of a full DELETE/recreate.

6. The locker PATCH endpoint refuses to edit reference-locked entries. Workaround: POST a new entry, then PUT the deployment target to reference the new entry's UUID. Old entry remains but is unreachable.

7. Fleet's import workflow auto-rewrites vcUsername on retry. Manual edits to set vcUsername = administrator@vsphere.local will be overwritten by an auto-generated svc-ops-vc-<uuid>-<uuid>-1@vsphere.local on the next retry. Don't fight it — create the auto-generated account in vSphere SSO instead.

8. Propagate to children is non-optional for the svc-ops account. Without it, the account can RetrieveServiceContent but cannot read child objects (datacenters, clusters, datastores). vim.fault.NoPermission surfaces at the inventory traversal step, not at login.

- 9. Pick the right "Administrator" role.** vSphere has multiple roles with "Administrator" in the name (e.g., "Content Library Registry administrator (sample)"). The one needed is the plain "Administrator" — usually near the top of the dropdown.
- 10. trust-manager Bundle auto-sync is fast.** Patching the source Secret propagates to all-namespace ConfigMaps within seconds; no manual re-sync needed. Verify via the Bundle's `status.conditions[?(@.type=="Synced")].status` field.
- 11. The svc-ops account UUID pattern is reproducible.** The first UUID is the env UUID's first 8-9 chars in some builds, but in 9.0.1 it's an opaque generated value. Capture it from the deployment target's `vcUsername` field after triggering one retry — that's the only reliable way to get the exact name needed for SSO user creation.

11. Document Information

Field	Value
Title	Fleet ↔ vCenter Trust Cascade Repair — Multi-Layer Recovery After vCenter Cert Rotation
Document ID	VC-RCA-FLEET-VC-CASCADE-20260518
Version	1.0
Issued	May 18, 2026
Author	Virtual Control LLC
Classification	Internal — Lab Environment
Related Documents	Fleet-Trust-Recovery-After-Cert-Rotation-KB402063-2026-05-17.md (Fleet-side cert rotation), Fleet-Operations-Integration-Recovery-RCA.md (broader integration RCA), VCF-Automation-Deployment-Troubleshooting-Report.md (the VCFA deploy this recovery unblocks)
Distribution	Lab notebook; do not redistribute externally without redacting credentials, hostnames, and the auto-generated svc-ops UUIDs

Revision History

Version	Date	Notes
1.0	May 18, 2026	Initial release. Standalone RCA + Runbook for the four-layer trust cascade triggered by vCenter cert rotation. Incident log captures the 2026-05-18 recovery (env <code>97dee43f-...</code> , new locker <code>c7288743-...</code> , svc-ops user <code>svc-ops-vc-1037eae3-...</code> , inventory resync completed at 2026-05-18T13:34Z). Documents the LCMCOMMON80062 → 80006 → 80015 → 80018 error code progression and the four trust stores that each need explicit repair (deployment target API, K8s platform-trust Secret, locker reference, SSO user + role).

Document End

See also: - [Fleet-Trust-Recovery-After-Cert-Rotation-KB402063-2026-05-17.md](#) — the Fleet-cert-rotation variant (companion runbook from the prior day) - [Fleet-Operations-Integration-Recovery-RCA.md](#) — the broader

Fleet/vcops integration RCA covering instance-limit deadlock and DB-direct cleanup - [VCF-Automation-Deployment-Troubleshooting-Report.md](#) — VCFA deployment troubleshooting (the deploy this recovery unblocks) - KB 402063 (public): <https://knowledge.broadcom.com/external/article/402063> - KB 421692 (public, related): <https://knowledge.broadcom.com/external/article/421692>